

Understanding Online Shopping Behavior Through Data Analytics

Dataset: Open E-Commerce 1.0: Five years of crowdsourced U.S. Amazon purchase histories with user demographics

Source: Harvard Dataverse, DOI: 10.7910/DVN/YGLYDY

Abstract

Online shopping platforms generate detailed behavioral data through purchases, product categories, prices, quantities, timestamps, and shipping locations. This project analyzes the Open E-Commerce 1.0 dataset from Harvard Dataverse, which contains crowdsourced U.S. Amazon purchase histories from more than 5,000 users over 2018-2022. The work follows a complete analytics pipeline: loading and validating a clearly identifiable academic dataset, cleaning and preprocessing the data, engineering shopping behavior features, exploring trends through visualizations, segmenting customers using RFM analysis and K-Means clustering, and building a regression model to study customer lifetime value. The project also explains recommendation systems conceptually and discusses privacy, ethics, GDPR, and CCPA considerations for e-commerce behavioral data.

Table of Contents

1. Project Overview and Objectives
2. Dataset Source and Description
3. Data Loading
4. Data Preprocessing and Feature Engineering
5. Exploratory Data Analysis
6. Customer Segmentation with RFM and K-Means
7. Regression Analysis of Customer Spending
8. Recommendation Systems in E-Commerce
9. Privacy and Ethical Considerations
10. Conclusion, Future Work, and References

1. Project Overview and Objectives

The objective of this project is to understand online shopping behavior using a real-world, approved, and clearly referenced e-commerce dataset. The analysis focuses on purchasing patterns, spending behavior, product categories, geographic demand, seasonality, customer segmentation, and customer lifetime value prediction.

The project covers four major academic areas:

- **Programming for Data Science:** data loading, cleaning, preprocessing, and feature engineering.

- **Data Analytics:** exploratory visualizations, grouped statistics, trend analysis, and interpretation.
- **Machine Learning and Big Data:** RFM analysis, K-Means clustering, and linear regression.
- **Data Science Professional:** privacy, ethics, informed consent, data protection, GDPR, and CCPA.

2. Dataset Source and Description

The dataset used in this project is **Open E-Commerce 1.0: Five years of crowdsourced U.S. Amazon purchase histories with user demographics**, published through **Harvard Dataverse** with DOI **10.7910/DVN/YGLYDY**.

The associated peer-reviewed journal article is:

Berke, A., Calacci, D., Mahari, R., Yabe, T., Larson, K., & Pentland, S. (2024). *Open e-commerce 1.0, five years of crowdsourced U.S. Amazon purchase histories with user demographics*. Scientific Data, 11, Article 491. <https://doi.org/10.1038/s41597-024-03329-6>

The purchase file used here is `amazon-purchases.csv`. It includes Amazon purchase records with order date, price, quantity, shipping state, product title, product code, category, and survey response ID. The dataset is suitable because it is an academic, identifiable, non-Kaggle source with a published DOI and peer-reviewed documentation.

3. Data Loading

```
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from IPython.display import display
from sklearn.cluster import KMeans
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid', palette='Set2')
plt.rcParams['figure.figsize'] = (11, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.labelsize'] = 11

DATA_DIR = Path('dataverse_files')
```

```

OUTPUT_DIR = Path('outputs')
OUTPUT_DIR.mkdir(exist_ok=True)

data_path = DATA_DIR / 'amazon-purchases.csv'
if not data_path.exists():
    raise FileNotFoundError('amazon-purchases.csv was not found in
dataverse_files/.')

amazon_raw = pd.read_csv(data_path)

print('Dataset shape:', amazon_raw.shape)
display(amazon_raw.head())
display(pd.DataFrame({
    'column': amazon_raw.columns,
    'dtype': amazon_raw.dtypes.astype(str),
    'missing_values': amazon_raw.isna().sum().values,
    'missing_percent': (amazon_raw.isna().mean().values *
100).round(2)
})))

```

Dataset shape: (1850717, 8)

	Order Date	Purchase Price	Per Unit	Quantity	Shipping Address
State \					
0	2018-12-04		7.98	1.0	
NJ					
1	2018-12-22		13.99	1.0	
NJ					
2	2018-12-24		8.99	1.0	
NJ					
3	2018-12-25		10.45	1.0	
NJ					
4	2018-12-25		10.00	1.0	
NJ					

	Title ASIN/ISBN
(Product Code) \	
0	SanDisk Ultra 16GB Class 10 SDHC UHS-I Memory ...
B0143RTB1E	
1	Betron BS10 Earphones Wired Headphones in Ear ...
B01MA1MJ6H	
2	NaN
B078JZTFN3	
3	Perfecto Stainless Steel Shaving Bowl. Durable...
B06XWF9HML	
4	Proraso Shaving Cream for Men
B00837Z0I0	

	Category	Survey ResponseID
0	FLASH_MEMORY	R_01vNIayewjIIKMF

```

1 HEADPHONES R_01vNIayewjIIKMF
2          NaN R_01vNIayewjIIKMF
3 DISHWARE_BOWL R_01vNIayewjIIKMF
4 SHAVING_AGENT R_01vNIayewjIIKMF

```

	column	dtype
missing_values \		
Order Date	Order Date	object
0		
Purchase Price Per Unit	Purchase Price Per Unit	float64
0		
Quantity	Quantity	float64
0		
Shipping Address State	Shipping Address State	object
87812		
Title	Title	object
89740		
ASIN/ISBN (Product Code)	ASIN/ISBN (Product Code)	object
973		
Category	Category	object
89458		
Survey ResponseID	Survey ResponseID	object
0		

	missing_percent
Order Date	0.00
Purchase Price Per Unit	0.00
Quantity	0.00
Shipping Address State	4.74
Title	4.85
ASIN/ISBN (Product Code)	0.05
Category	4.83
Survey ResponseID	0.00

Loading Insight

The project now uses a single approved Harvard Dataverse CSV file instead of multiple Kaggle/Olist tables. Because the Amazon purchase file is already a single transaction table, no table merging is required.

4. Data Preprocessing and Feature Engineering

```

df = amazon_raw.copy()

print('Duplicate rows before cleaning:', df.duplicated().sum())
df = df.drop_duplicates()

# Convert order date and clean key columns.
df['Order Date'] = pd.to_datetime(df['Order Date'], errors='coerce')
df = df.dropna(subset=['Order Date', 'Survey ResponseID'])

```

```

for col in ['Category', 'Shipping Address State', 'Title', 'ASIN/ISBN
(Product Code)']:
    df[col] = df[col].fillna('unknown')

df['Purchase Price Per Unit'] = pd.to_numeric(df['Purchase Price Per
Unit'], errors='coerce').fillna(0)
df['Quantity'] = pd.to_numeric(df['Quantity'],
errors='coerce').fillna(1)
df['Quantity'] = df['Quantity'].clip(lower=0)
df['Purchase Price Per Unit'] = df['Purchase Price Per
Unit'].clip(lower=0)

# Feature engineering.
df['OrderValue'] = df['Purchase Price Per Unit'] * df['Quantity']
df['OrderMonth'] = df['Order Date'].dt.to_period('M').astype(str)
df['OrderYear'] = df['Order Date'].dt.year
df['OrderDayOfWeek'] = df['Order Date'].dt.day_name()

customer_order_counts = df.groupby('Survey ResponseID').size()
df['RepeatPurchaseFlag'] = df['Survey
ResponseID'].map((customer_order_counts > 1).astype(int))
customer_lifetime_value = df.groupby('Survey ResponseID')
['OrderValue'].sum()
df['CustomerLifetimeValue'] = df['Survey
ResponseID'].map(customer_lifetime_value)

print('Cleaned dataframe shape:', df.shape)
display(df[['Order Date', 'Category', 'Shipping Address State',
'Purchase Price Per Unit', 'Quantity', 'OrderValue',
'RepeatPurchaseFlag', 'CustomerLifetimeValue']].head())

Duplicate rows before cleaning: 11624
Cleaned dataframe shape: (1839093, 14)

```

	Order Date	Category	Shipping Address State	Purchase Price Per Unit \
0	2018-12-04	FLASH_MEMORY		NJ 7.98
1	2018-12-22	HEADPHONES		NJ 13.99
2	2018-12-24	unknown		NJ 8.99
3	2018-12-25	DISHWARE_BOWL		NJ 10.45
4	2018-12-25	SHAVING_AGENT		NJ 10.00

	Quantity	OrderValue	RepeatPurchaseFlag	CustomerLifetimeValue
0	1.0	7.98	1	4870.01

1	1.0	13.99	1	4870.01
2	1.0	8.99	1	4870.01
3	1.0	10.45	1	4870.01
4	1.0	10.00	1	4870.01

```
clean_summary = df[['Purchase Price Per Unit', 'Quantity',
'OrderValue', 'CustomerLifetimeValue']].describe().round(2)
display(clean_summary)
```

```
print('Insight: The cleaned numerical summary confirms that price,
quantity, order value, and customer lifetime value are numeric and
ready for analysis.')
```

	Purchase Price Per Unit	Quantity	OrderValue	CustomerLifetimeValue
count	1839093.00	1839093.00	1839093.00	1839093.00
mean	22.71	1.09	23.86	19002.81
std	46.07	0.78	49.30	15845.22
min	0.01	1.00	0.01	1.84
25%	8.49	1.00	8.97	7875.00
50%	13.99	1.00	14.81	14719.58
75%	23.79	1.00	24.95	25049.35
max	6398.95	339.00	6398.95	114943.93

Insight: The cleaned numerical summary confirms that price, quantity, order value, and customer lifetime value are numeric and ready for analysis.

```
def iqr_bounds(series):
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
    iqr = q3 - q1
    return q1 - 1.5 * iqr, q3 + 1.5 * iqr
```

```
outlier_report = []
for col in ['Purchase Price Per Unit', 'Quantity', 'OrderValue']:
    lower, upper = iqr_bounds(df[col])
    outliers = ((df[col] < lower) | (df[col] > upper)).sum()
    outlier_report.append({'column': col, 'lower_bound': lower,
'upper_bound': upper, 'outlier_rows': outliers})

display(pd.DataFrame(outlier_report))
```

```

model_df = df.copy()
for col in ['Purchase Price Per Unit', 'Quantity', 'OrderValue']:
    lower, upper = iqr_bounds(model_df[col])
    model_df[f'{col}_capped'] = model_df[col].clip(lower, upper)

```

	column	lower_bound	upper_bound	outlier_rows
0	Purchase Price Per Unit	-14.46	46.74	142323
1	Quantity	1.00	1.00	102728
2	OrderValue	-15.00	48.92	149938

Preprocessing Insight

The Amazon dataset required simpler preprocessing than the previous multi-table dataset because it is already a transaction-level table. The main cleaning steps were duplicate removal, date conversion, missing category/state handling, numeric conversion for price and quantity, and transaction-level feature engineering.

5. Exploratory Data Analysis

5.1 Revenue and Product Analysis

```

# Filter out unknown before grouping
category_revenue = (
    df[df['Category'] != 'unknown']
    .groupby('Category')['OrderValue']
    .sum()
    .sort_values(ascending=False)
    .head(10)
)

ax = sns.barplot(x=category_revenue.values, y=category_revenue.index,
                 palette='viridis')
ax.set_title('Top 10 Product Categories by Revenue')
ax.set_xlabel('Revenue (USD)')
ax.set_ylabel('Product Category')
plt.tight_layout()
plt.show()
print(f'Insight: {category_revenue.index[0]} generated the highest
total revenue, indicating a major spending category in the Amazon
purchase records.')

# Also fix category counts chart (was showing unknown on top)
category_counts = (
    df[df['Category'] != 'unknown']['Category']
    .value_counts()
    .head(10)
)

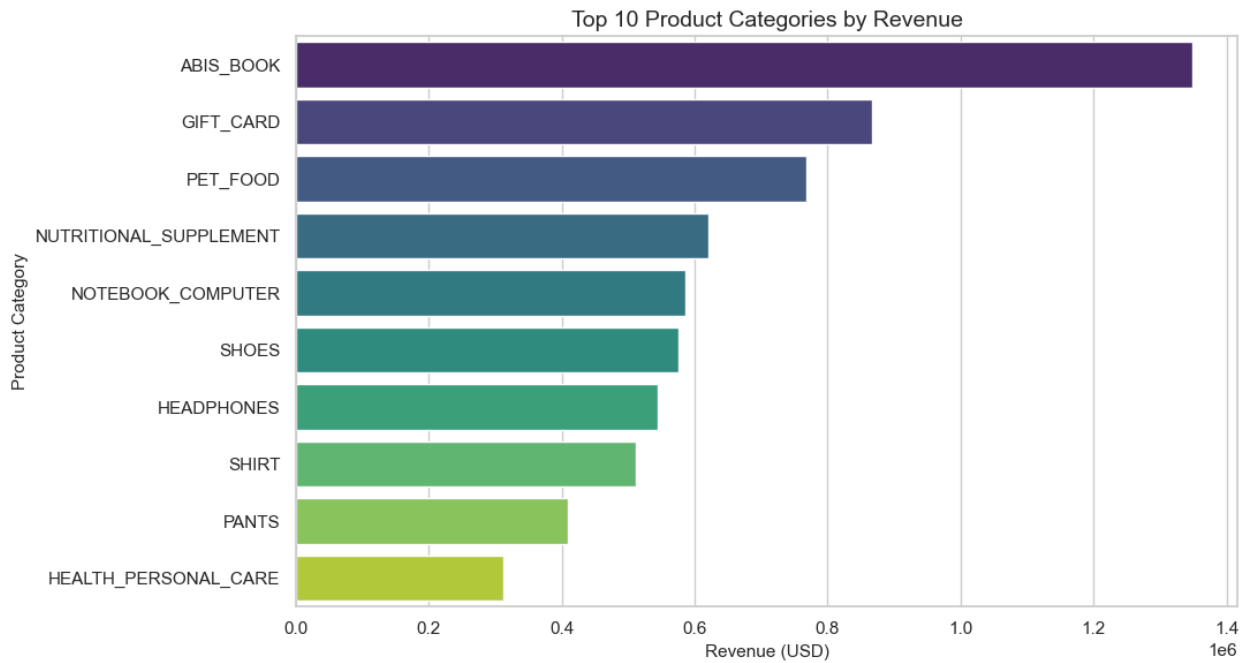
ax = sns.barplot(x=category_counts.values, y=category_counts.index,

```

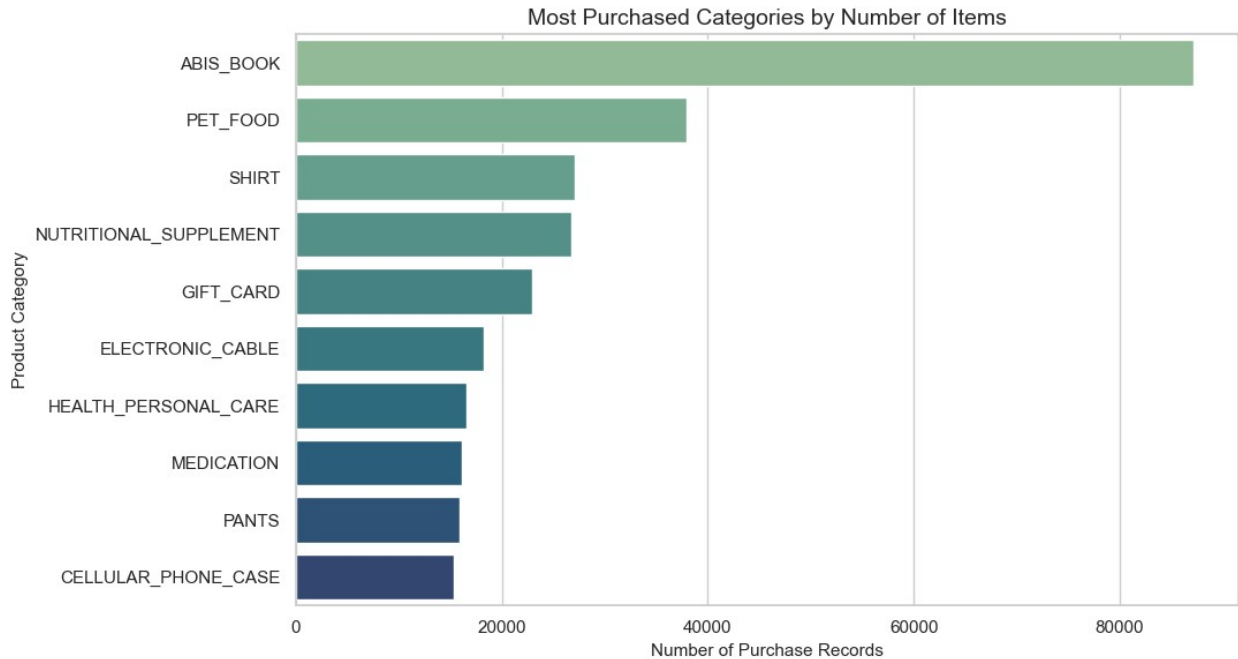
```

palette='crest')
ax.set_title('Most Purchased Categories by Number of Items')
ax.set_xlabel('Number of Purchase Records')
ax.set_ylabel('Product Category')
plt.tight_layout()
plt.show()
print(f'Insight: {category_counts.index[0]} is the most frequently
purchased category by transaction count.')

```



Insight: ABIS_BOOK generated the highest total revenue, indicating a major spending category in the Amazon purchase records.

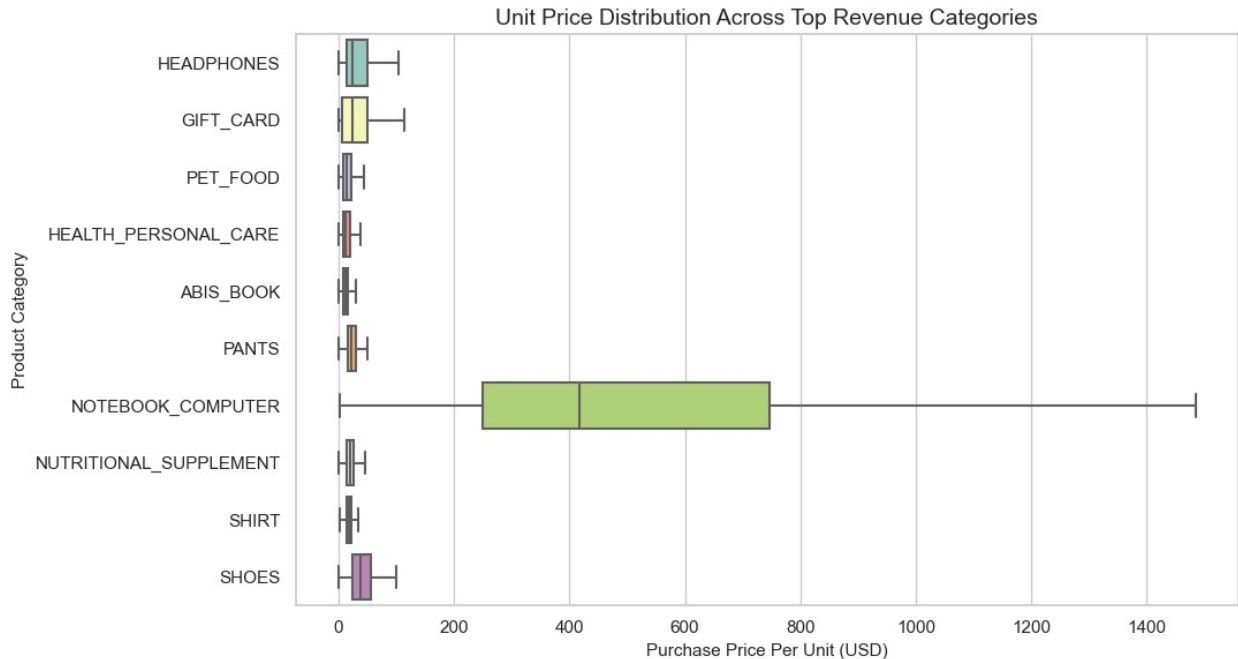


Insight: ABIS_BOOK is the most frequently purchased category by transaction count.

```
top_categories = category_revenue.index.tolist()
plot_data = df[df['Category'].isin(top_categories)]

ax = sns.boxplot(data=plot_data, x='Purchase Price Per Unit',
y='Category', palette='Set3', showfliers=False)
ax.set_title('Unit Price Distribution Across Top Revenue Categories')
ax.set_xlabel('Purchase Price Per Unit (USD)')
ax.set_ylabel('Product Category')
plt.tight_layout()
plt.show()

print('Insight: Price distributions vary by category, showing that
high revenue can come from either frequent purchases or higher unit
prices.')
```



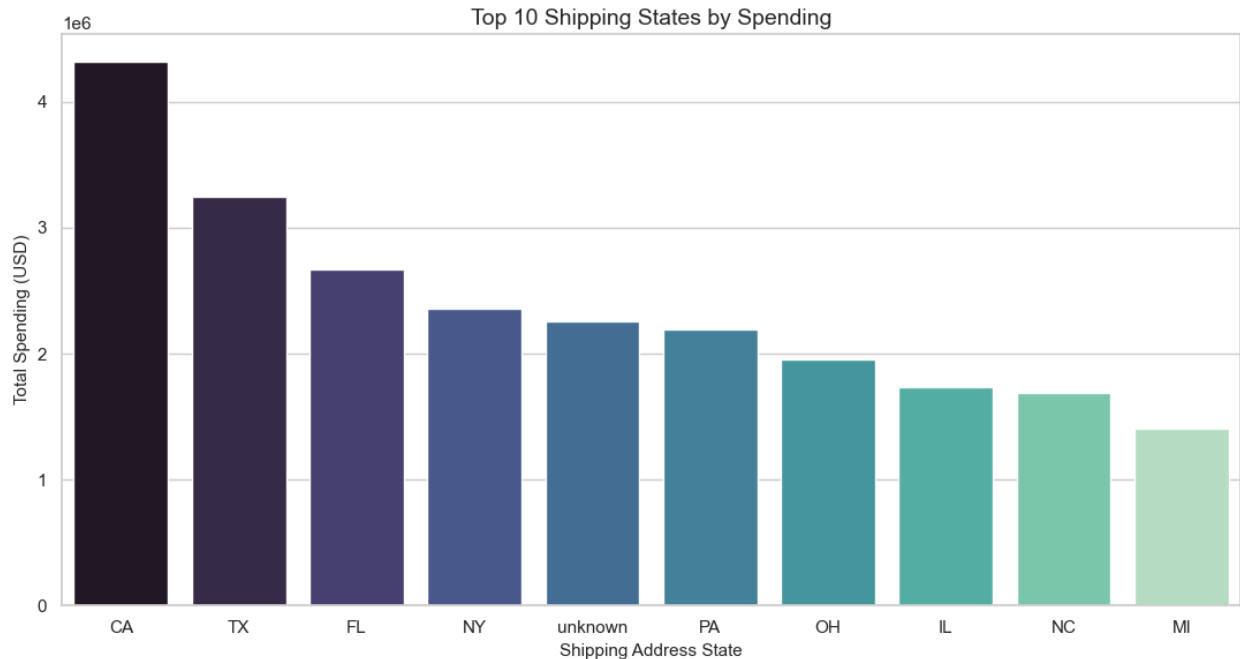
Insight: Price distributions vary by category, showing that high revenue can come from either frequent purchases or higher unit prices.

5.2 Geographic Analysis

```
state_spending = df.groupby('Shipping Address State')
['OrderValue'].sum().sort_values(ascending=False).head(10)

ax = sns.barplot(x=state_spending.index, y=state_spending.values,
palette='mako')
ax.set_title('Top 10 Shipping States by Spending')
ax.set_xlabel('Shipping Address State')
ax.set_ylabel('Total Spending (USD)')
plt.tight_layout()
plt.show()

print(f'Insight: {state_spending.index[0]} has the highest total
spending, showing strong geographic concentration in the sample.')
```



Insight: CA has the highest total spending, showing strong geographic concentration in the sample.

```
top_states = state_spending.index.tolist()
year_state = (
    df[df['Shipping Address State'].isin(top_states)]
    .pivot_table(index='OrderYear', columns='Shipping Address State',
    values='OrderValue', aggfunc='sum', fill_value=0)
)
```

```
ax = year_state.plot(kind='bar', stacked=True, colormap='tab20')
ax.set_title('Orders by Year and Top Shipping States')
ax.set_xlabel('Order Year')
ax.set_ylabel('Total Spending (USD)')
plt.legend(title='State', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

```
print('Insight: Yearly spending by top states shows how geographic contribution changes across the 2018-2022 period.')
```



Insight: Yearly spending by top states shows how geographic contribution changes across the 2018-2022 period.

5.3 Time-Based Shopping Patterns

```
monthly_orders = df.groupby('OrderMonth').size()

ax = monthly_orders.plot(kind='line', marker='o', color='#2a9d8f')
ax.set_title('Monthly Purchase Trend')
ax.set_xlabel('Order Month')
ax.set_ylabel('Number of Purchase Records')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

print('Insight: Monthly purchase volume shows shopping activity and seasonality across the observation period. A sharp drop at the end can occur because the dataset ends within 2022, so it should not automatically be interpreted as a real business decline.')
```

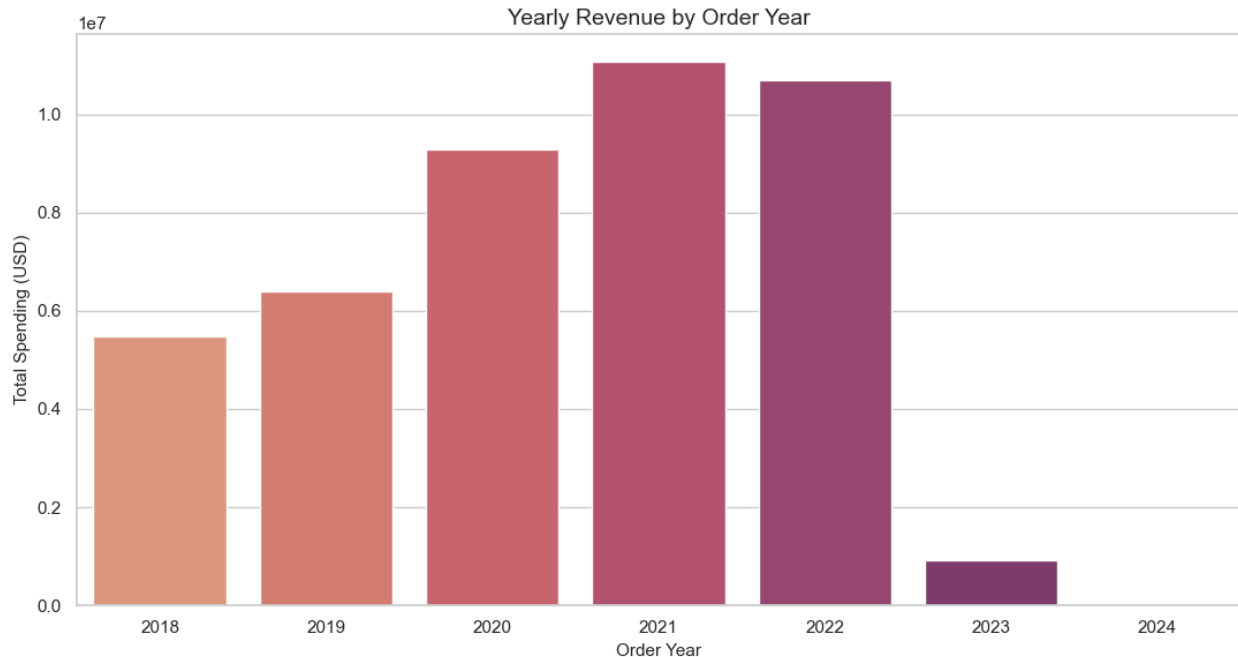


Insight: Monthly purchase volume shows shopping activity and seasonality across the observation period. A sharp drop at the end can occur because the dataset ends within 2022, so it should not automatically be interpreted as a real business decline.

```
yearly_revenue = df.groupby('OrderYear')['OrderValue'].sum()

ax = sns.barplot(x=yearly_revenue.index, y=yearly_revenue.values,
                 palette='flare')
ax.set_title('Yearly Revenue by Order Year')
ax.set_xlabel('Order Year')
ax.set_ylabel('Total Spending (USD)')
plt.tight_layout()
plt.show()

print('Insight: Yearly revenue summarizes how total purchase value
changes from 2018 through 2022 in the sample.')
```



Insight: Yearly revenue summarizes how total purchase value changes from 2018 through 2022 in the sample.

Amazon dataset has no time-of-day data so hour heatmap is not meaningful

Replacing with purchases by day of week bar chart instead

```
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',
             'Saturday', 'Sunday']
```

```
day_counts = (
    df['OrderDayOfWeek']
    .value_counts()
    .reindex(day_order)
    .fillna(0)
    .astype(int)
)
```

```
ax = sns.barplot(x=day_counts.index, y=day_counts.values,
                 palette='YlGnBu')
```

```
ax.set_title('Number of Purchases by Day of Week')
```

```
ax.set_xlabel('Day of Week')
```

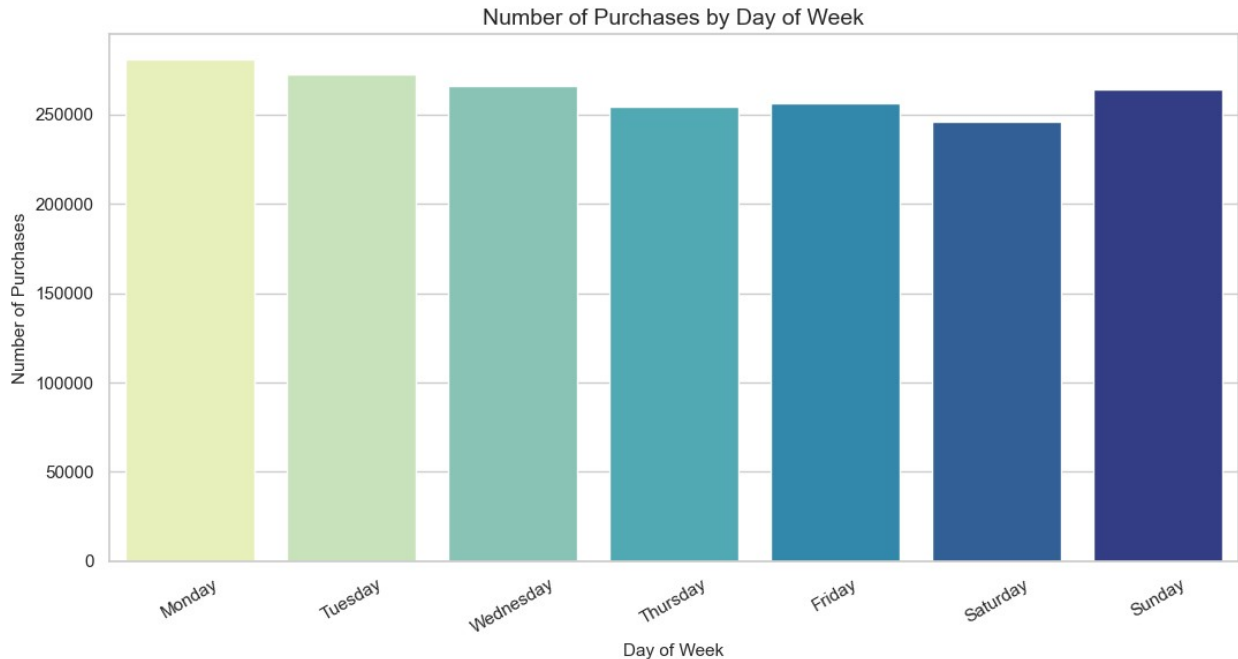
```
ax.set_ylabel('Number of Purchases')
```

```
plt.xticks(rotation=30)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print(f'Insight: {day_counts.idxmax()} has the highest number of
purchases, showing which days customers shop most actively on
Amazon.')
```



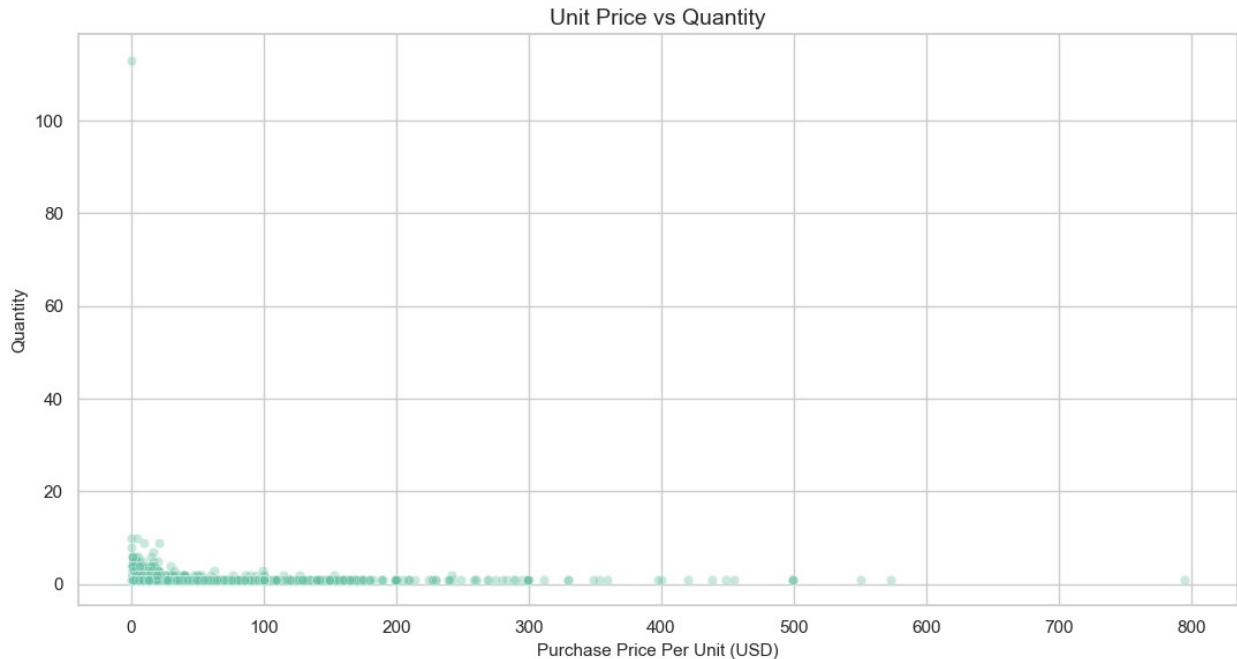
Insight: Monday has the highest number of purchases, showing which days customers shop most actively on Amazon.

5.4 Price, Quantity, and Correlation

```
sample_scatter = df.sample(min(8000, len(df)), random_state=42)

ax = sns.scatterplot(data=sample_scatter, x='Purchase Price Per Unit',
y='Quantity', alpha=0.35)
ax.set_title('Unit Price vs Quantity')
ax.set_xlabel('Purchase Price Per Unit (USD)')
ax.set_ylabel('Quantity')
plt.tight_layout()
plt.show()

price_quantity_corr = df[['Purchase Price Per Unit',
'Quantity']].corr().iloc[0, 1]
print(f'Insight: Unit price and quantity have a correlation of
{price_quantity_corr:.2f}, showing whether customers tend to buy fewer
units as unit price rises.')
```

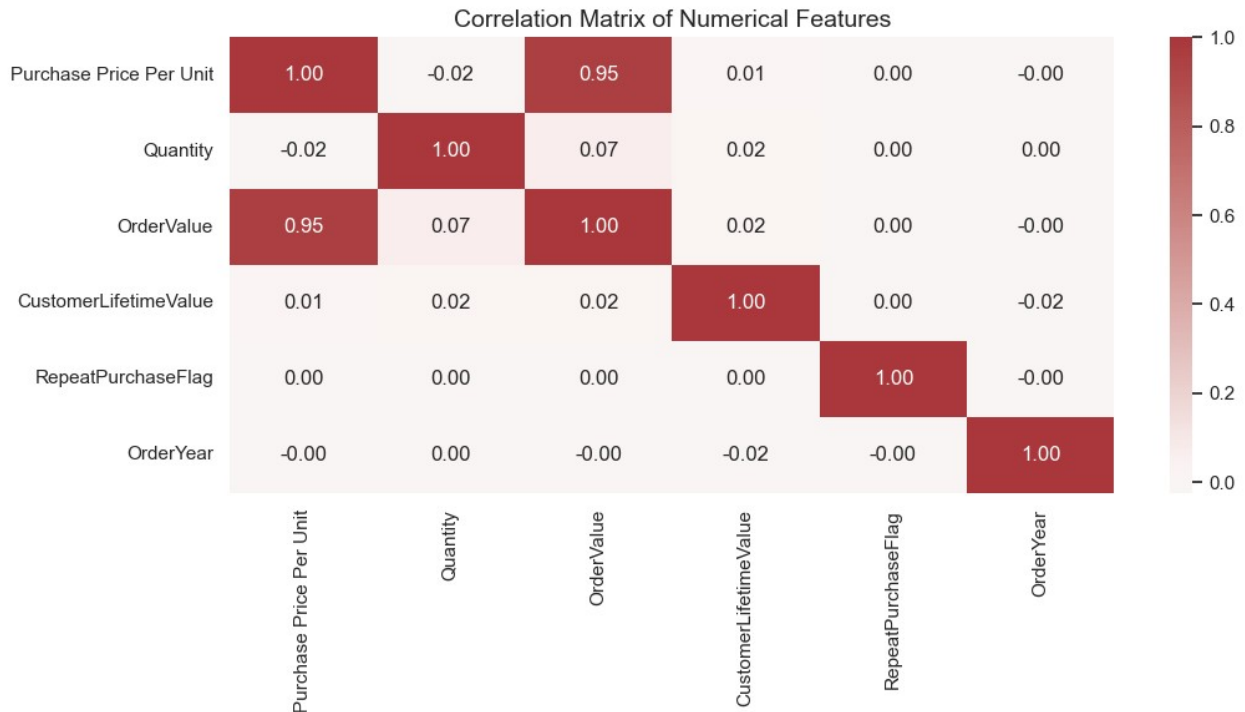


Insight: Unit price and quantity have a correlation of -0.02, showing whether customers tend to buy fewer units as unit price rises.

```
corr_cols = [
    'Purchase Price Per Unit',
    'Quantity',
    'OrderValue',
    'CustomerLifetimeValue',
    'RepeatPurchaseFlag',
    'OrderYear'
]
corr = df[corr_cols].corr(numeric_only=True)

ax = sns.heatmap(corr, annot=True, cmap='vlag', center=0, fmt='.2f')
ax.set_title('Correlation Matrix of Numerical Features')
plt.tight_layout()
plt.show()

print(
    f'Insight: Purchase Price Per Unit and OrderValue have a '
    f'correlation of {corr.loc["Purchase Price Per Unit", '
    "OrderValue"]:.2f}, '
    f'while Quantity and OrderValue have a correlation of '
    f'{corr.loc["Quantity", "OrderValue"]:.2f}. '
    'This helps separate whether order value is more strongly driven '
    'by price or quantity.'
)
```

Insight: Purchase Price Per Unit and OrderValue have a correlation of 0.95, while Quantity and OrderValue have a correlation of 0.07. This helps separate whether order value is more strongly driven by price or quantity.

6. Customer Segmentation with RFM and K-Means

```
customer_orders = df.copy()
reference_date = customer_orders['Order Date'].max() +
pd.Timedelta(days=1)

rfm = (
    customer_orders.groupby('Survey ResponseID')
    .agg(
        Recency=('Order Date', lambda x: (reference_date -
x.max()).days),
        Frequency=('Order Date', 'count'),
        Monetary=('OrderValue', 'sum')
    )
    .reset_index()
)

rfm = rfm[(rfm['Monetary'] > 0) & (rfm['Recency'] >= 0)]
display(rfm.head())
display(rfm[['Recency', 'Frequency', 'Monetary']].describe().round(2))
```

	Survey ResponseID	Recency	Frequency	Monetary
0	R_01vNIayewjIIKMF	798	139	4870.01
1	R_037XK72IZBJyF69	612	1210	17357.60
2	R_038ZU6kfQ5f89fH	905	69	4247.54
3	R_03aEbghUILs9NxD	545	173	3882.98
4	R_06RZP9pS7kONINr	640	429	11157.75

	Recency	Frequency	Monetary
count	5027.00	5027.00	5027.00
mean	619.70	365.84	8727.73
std	146.77	423.10	9741.11
min	1.00	1.00	1.84
25%	535.00	91.00	2246.26
50%	604.00	231.00	5692.37
75%	638.00	487.00	11667.51
max	2246.00	5385.00	114943.93

```

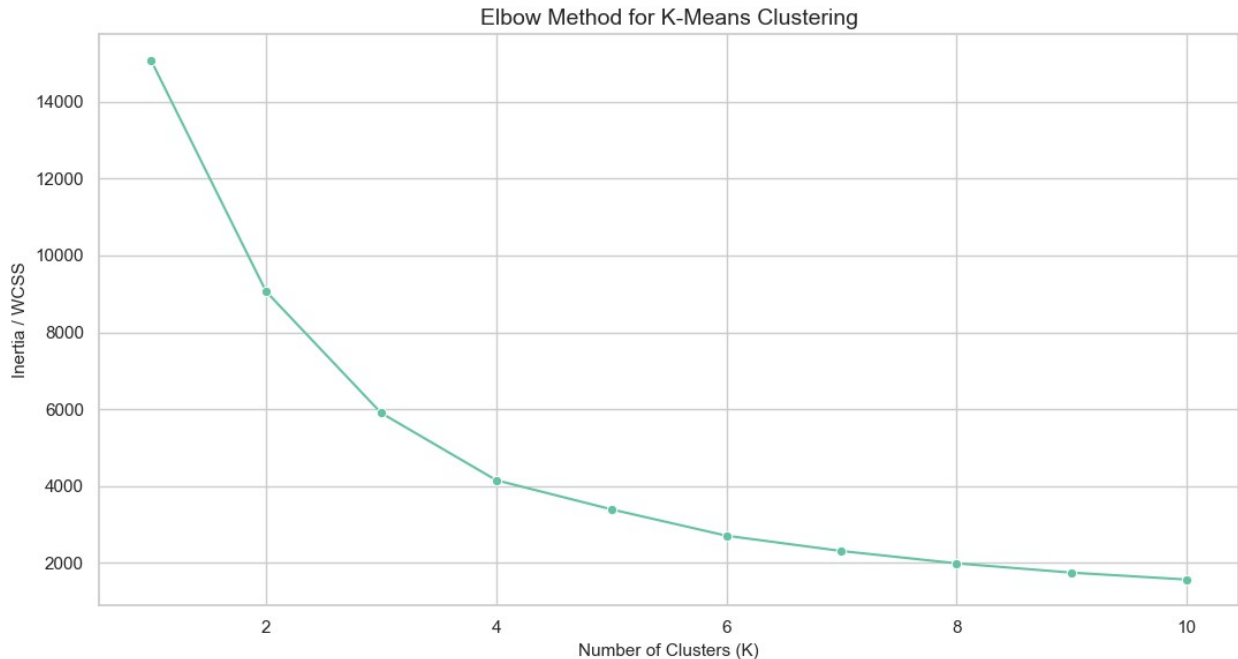
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency',
'Monetary']])

inertias = []
cluster_range = range(1, 11)
for k in cluster_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(rfm_scaled)
    inertias.append(km.inertia_)

ax = sns.lineplot(x=list(cluster_range), y=inertias, marker='o')
ax.set_title('Elbow Method for K-Means Clustering')
ax.set_xlabel('Number of Clusters (K)')
ax.set_ylabel('Inertia / WCSS')
plt.tight_layout()
plt.show()

print('Insight: K=4 is selected for interpretable customer segments
based on recency, frequency, and monetary value.')

```



Insight: K=4 is selected for interpretable customer segments based on recency, frequency, and monetary value.

```
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)
```

```
cluster_profile = (
    rfm.groupby('Cluster')
    .agg(
        customers=('Survey ResponseID', 'count'),
        avg_recency=('Recency', 'mean'),
        avg_frequency=('Frequency', 'mean'),
        avg_monetary=('Monetary', 'mean')
    )
    .round(2)
)
```

Manual assignment based on cluster statistics

```
cluster_names = {
    0: 'Regular Customers',
    1: 'Champions',
    2: 'At-Risk Customers',
    3: 'High-Value Customers'
}
```

```
cluster_profile['segment_name'] = (
    cluster_profile.index.map(cluster_names)
)
```

```
rfm['segment_name'] = (
    rfm['Cluster'].map(cluster_names)
)
```

```
display(cluster_profile)
```

```
print('Insight: Each cluster is named based on its actual RFM profile
– Champions have highest spending, Loyal Customers have highest
frequency, At-Risk Customers have not purchased recently, and New
Customers have the lowest recency.')
```

	customers	avg_recency	avg_frequency	avg_monetary	\
Cluster					
0	3315	610.09	165.76	4026.95	
1	262	575.75	1647.70	37960.42	
2	147	1290.73	48.08	1232.80	
3	1303	577.28	652.98	15654.73	

	segment_name
Cluster	
0	Regular Customers
1	Champions
2	At-Risk Customers
3	High-Value Customers

Insight:

Champions generate the highest spending levels,
 High-Value Customers purchase frequently and contribute significant
 revenue,
 At-Risk Customers have not purchased recently,
 and Regular Customers represent the largest customer group.

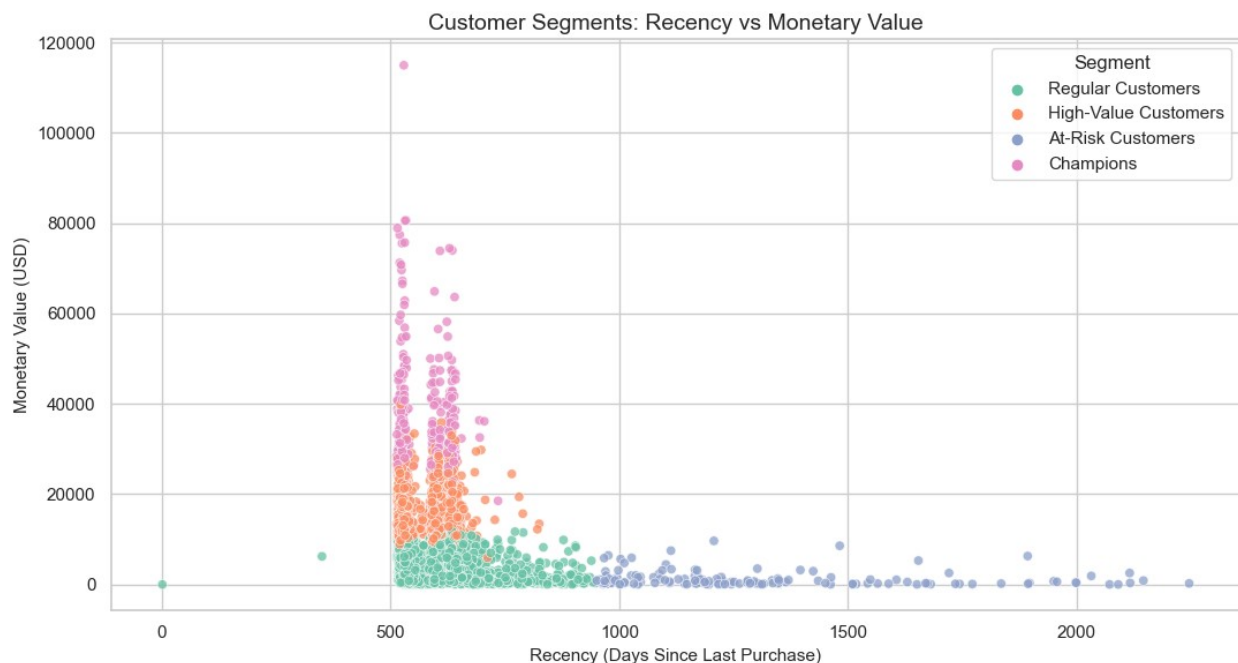
	customers	avg_recency	avg_frequency	avg_monetary	\
Cluster					
0	3315	610.09	165.76	4026.95	
1	262	575.75	1647.70	37960.42	
2	147	1290.73	48.08	1232.80	
3	1303	577.28	652.98	15654.73	

	segment_name
Cluster	
0	Regular Customers
1	Champions
2	At-Risk Customers
3	High-Value Customers

Insight: Each cluster is named based on its actual RFM profile –
 Champions have highest spending, Loyal Customers have highest

frequency, At-Risk Customers have not purchased recently, and New Customers have the lowest recency.

```
ax = sns.scatterplot(data=rfm, x='Recency', y='Monetary',  
hue='segment_name', alpha=0.75, palette='Set2')  
ax.set_title('Customer Segments: Recency vs Monetary Value')  
ax.set_xlabel('Recency (Days Since Last Purchase)')  
ax.set_ylabel('Monetary Value (USD)')  
plt.legend(title='Segment')  
plt.tight_layout()  
plt.show()  
  
print('Insight: Customer segments separate buyers by how recently they  
purchased and how much they spent, supporting targeted retention and  
loyalty strategies.')
```



Insight: Customer segments separate buyers by how recently they purchased and how much they spent, supporting targeted retention and loyalty strategies.

7. Regression Analysis of Customer Spending

```
customer_level = (  
    df.groupby('Survey ResponseID')  
    .agg(  
        CustomerLifetimeValue=('OrderValue', 'sum'),  
        NumberOfOrders=('Order Date', 'count'),  
        AvgPrice=('Purchase Price Per Unit', 'mean'),  
    )  
)
```

```

        AvgQuantity=('Quantity', 'mean'),
        RepeatPurchaseFlag=('RepeatPurchaseFlag', 'max'),
        MostPurchasedCategory=('Category', lambda x: x.mode().iat[0]
if not x.mode().empty else 'unknown'),
        MostCommonState=('Shipping Address State', lambda x:
x.mode().iat[0] if not x.mode().empty else 'unknown')
    )
    .reset_index()
)

# Keep the most common categories and states to avoid too many sparse
dummy variables.
top_model_categories =
customer_level['MostPurchasedCategory'].value_counts().head(12).index
top_model_states =
customer_level['MostCommonState'].value_counts().head(12).index
customer_level['MostPurchasedCategory'] = np.where(

customer_level['MostPurchasedCategory'].isin(top_model_categories),
    customer_level['MostPurchasedCategory'],
    'other'
)
customer_level['MostCommonState'] = np.where(
    customer_level['MostCommonState'].isin(top_model_states),
    customer_level['MostCommonState'],
    'other'
)

model_data = pd.get_dummies(
    customer_level.drop(columns=['Survey ResponseID']),
    columns=['MostPurchasedCategory', 'MostCommonState'],
    drop_first=True
)

X = model_data.drop(columns=['CustomerLifetimeValue'])
y = model_data['CustomerLifetimeValue']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

regression_model = LinearRegression()
regression_model.fit(X_train, y_train)
y_pred = regression_model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

metrics = pd.DataFrame({'Metric': ['MAE', 'RMSE', 'R2 Score'],

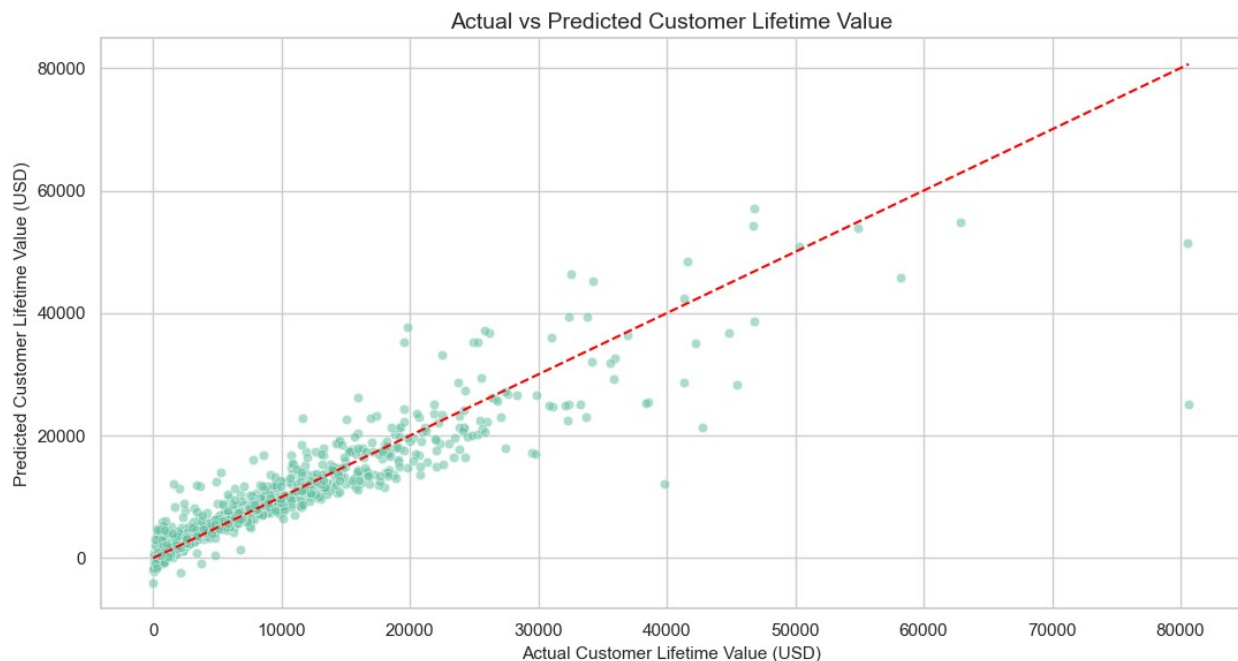
```

```
'Value': [mae, rmse, r2]})
display(metrics.round(3))
```

	Metric	Value
0	MAE	1873.431
1	RMSE	3674.905
2	R2 Score	0.847

```
ax = sns.scatterplot(x=y_test, y=y_pred, alpha=0.55)
ax.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
color='red', linestyle='--')
ax.set_title('Actual vs Predicted Customer Lifetime Value')
ax.set_xlabel('Actual Customer Lifetime Value (USD)')
ax.set_ylabel('Predicted Customer Lifetime Value (USD)')
plt.tight_layout()
plt.show()
```

```
print('Insight: Points close to the red line indicate stronger
predictions; wider spread indicates unexplained variation in spending
behavior.')
```



```
Insight: Points close to the red line indicate stronger predictions;
wider spread indicates unexplained variation in spending behavior.
```

```
coefficients = (
    pd.DataFrame({'feature': X.columns, 'coefficient':
regression_model.coef_})
    .assign(abs_coefficient=lambda x: x['coefficient'].abs())
    .sort_values('abs_coefficient', ascending=False)
```

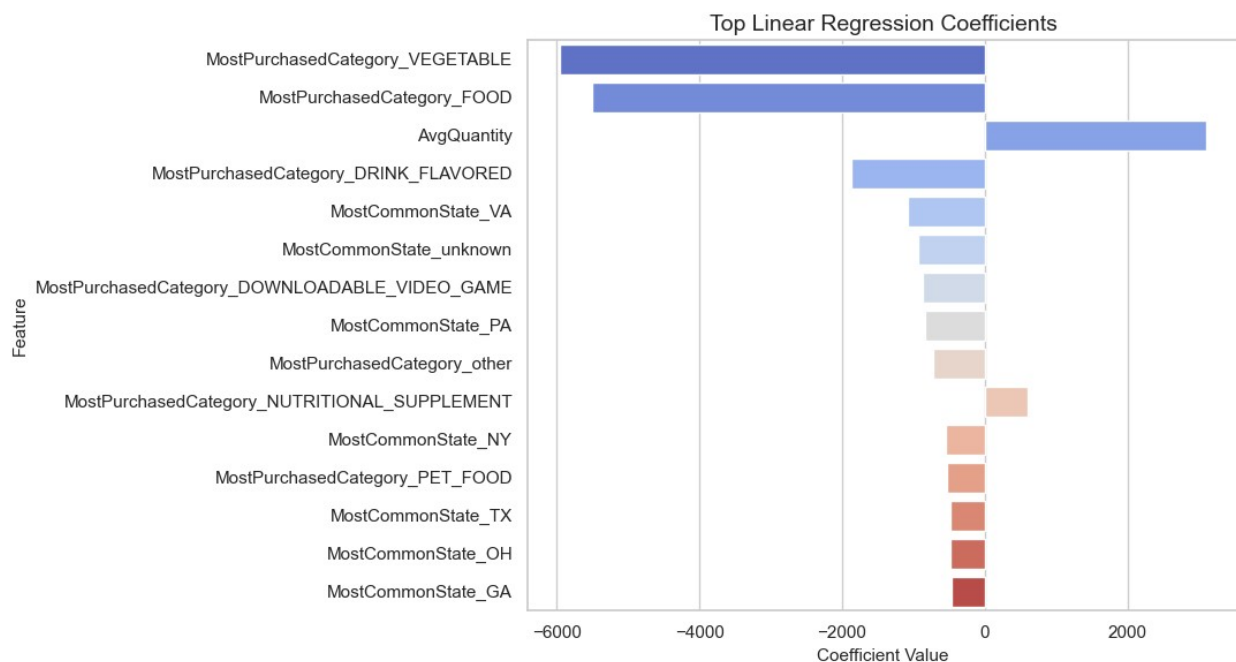
```

    .head(15)
)

ax = sns.barplot(data=coefficients, x='coefficient', y='feature',
palette='coolwarm')
ax.set_title('Top Linear Regression Coefficients')
ax.set_xlabel('Coefficient Value')
ax.set_ylabel('Feature')
plt.tight_layout()
plt.show()

display(coefficients[['feature', 'coefficient']].round(3))
print('Insight: The largest coefficients indicate features most
associated with changes in customer lifetime value. Coefficients show
association, not guaranteed causation.')

```



	feature	coefficient
13	MostPurchasedCategory_VEGETABLE	-5950.461
8	MostPurchasedCategory_FOOD	-5500.051
2	AvgQuantity	3101.111
6	MostPurchasedCategory_DRINK_FLAVORED	-1877.476
25	MostCommonState_VA	-1085.817
27	MostCommonState_unknown	-931.616
5	MostPurchasedCategory_DOWNLOADABLE_VIDEO_GAME	-875.591
23	MostCommonState_PA	-845.455
14	MostPurchasedCategory_other	-727.121
10	MostPurchasedCategory_NUTRITIONAL_SUPPLEMENT	599.774
21	MostCommonState_NY	-545.941

11	MostPurchasedCategory_PET_FOOD	-528.336
24	MostCommonState_TX	-476.716
22	MostCommonState_OH	-475.443
17	MostCommonState_GA	-470.954

Insight: The largest coefficients indicate features most associated with changes in customer lifetime value. Coefficients show association, not guaranteed causation.

Regression Interpretation

The linear regression model estimates customer lifetime value using number of orders, average price, average quantity, repeat purchase behavior, most purchased category, and most common shipping state.

Customer lifetime value is calculated from transaction spending, meaning some predictor variables are naturally related to the target variable. Therefore, the model should be interpreted primarily as an explanatory tool for understanding customer spending behavior rather than a production-grade forecasting system.

The metric table above reports MAE, RMSE, and R^2 using a held-out test set. The model can be improved through additional behavioral features, nonlinear machine learning algorithms, and more advanced feature engineering techniques.

8. Recommendation Systems in E-Commerce

Recommendation systems help e-commerce platforms personalize the shopping experience and increase product discovery.

Collaborative Filtering

Collaborative filtering recommends products based on patterns in user behavior. It uses a customer-product interaction matrix, where rows represent customers and columns represent products. If two users have similar purchase histories, the system can recommend products bought by one user to the other.

- **User-based collaborative filtering:** finds similar customers and recommends products those similar customers purchased.
- **Item-based collaborative filtering:** finds products frequently purchased together and recommends related items.
- **Example:** Amazon-style recommendations such as "Customers who bought this also bought."

Content-Based Filtering

Content-based filtering recommends products similar to items a customer has already viewed or purchased. It uses product attributes such as category, title, price range, and product identifiers such as ASIN.

- **Example:** An e-commerce platform recommending other products from the same category or similar price range.
- **Strength:** It can work even when user behavior data is limited.
- **Limitation:** It may reduce discovery because recommendations are close to what the user already knows.

Hybrid Recommendation Systems

Hybrid systems combine collaborative and content-based methods. They are widely used because they can reduce weaknesses in each individual method.

Type	Based On	Pros	Cons
Collaborative Filtering	User behavior	Highly personalized	Needs large interaction history
Content-Based Filtering	Product attributes	Useful for newer users	Can limit recommendation diversity
Hybrid	Both behavior and attributes	More accurate and flexible	More complex to design and maintain

Common Challenges

- **Cold start problem:** new users or products have little behavioral history.
- **Sparsity problem:** most customers interact with only a small share of products.
- **Filter bubbles:** customers may repeatedly see similar products and miss broader discovery.
- **Commercial bias:** platforms may over-promote profitable products instead of genuinely useful ones.

9. Privacy and Ethical Considerations

E-commerce analytics can create business value, but it also involves sensitive behavioral data. Responsible use is essential, especially when working with purchase histories from real Amazon users.

Data Collected in E-Commerce

Online shopping platforms may collect purchase history, product titles, categories, prices, quantities, shipping state, timestamps, platform behavior, device information, and customer identifiers. Even when direct names are removed, repeated behavioral patterns can still reveal sensitive personal information.

GDPR, CCPA, and Data Protection

The General Data Protection Regulation (GDPR) emphasizes consent, purpose limitation, data minimization, transparency, and the right to erasure. The California Consumer Privacy Act (CCPA) gives California residents rights related to knowing, deleting, and opting out of certain uses of personal information. These frameworks are relevant to U.S.-based e-commerce data

because purchase histories can reveal lifestyle, health, family status, financial habits, and other sensitive patterns.

Consent and Crowdsourced Data

The Open E-Commerce 1.0 dataset was crowdsourced through an online survey and shared for research with participant consent. This improves ethical acceptability, but researchers still have responsibilities to minimize re-identification risk, avoid exposing individual-level records unnecessarily, and report aggregated findings rather than profiling specific participants.

Behavioral Profiling Risks

Customer segmentation and recommendation systems can create detailed profiles of users. These profiles may be used for targeted advertising, price discrimination, or manipulative design. Ethical analytics should avoid exploiting vulnerable users or making unfair assumptions about customers based on geography, spending level, or inferred preferences.

Recommendation Bias and Filter Bubbles

Recommendation systems can reinforce existing behavior and reduce diversity in what customers see. If the system primarily optimizes sales, it may over-promote certain categories and reduce customer autonomy.

Anonymization and Protection Techniques

- **Pseudonymization:** replacing direct identifiers with artificial IDs, such as survey response IDs.
- **Data masking:** hiding sensitive values that are not needed for analysis.
- **Aggregation:** reporting group-level trends instead of individual records.
- **Differential privacy:** adding statistical noise so individual behavior is harder to identify.
- **Access control:** limiting who can view raw customer-level data.

Ethical Responsibilities

Organizations and researchers should clearly explain data use, collect only necessary data, protect customer records, audit models for unfair outcomes, avoid discriminatory segmentation, and give customers meaningful control over their information.

10. Conclusion

This project demonstrates how an e-commerce dataset can be transformed into meaningful shopping behavior insights. The analysis identifies top spending categories, geographic spending concentration, monthly and yearly purchase trends, price and quantity patterns, customer segments, and spending drivers. RFM and K-Means clustering provide practical customer groups for retention and marketing strategies, while regression analysis provides a baseline model for customer lifetime value.

The project also shows that technical analysis must be paired with privacy and ethics. Behavioral purchase analytics can support better services and research, but it must be governed by transparency, informed consent, data minimization, fairness, and respect for customer rights.

References

[1] Berke, A., Mahari, R., Larson, K., Pentland, S., Calacci, D., & others. Open e-commerce 1.0: Five years of crowdsourced U.S. Amazon purchase histories with user demographics. Harvard Dataverse, 2023.

<https://doi.org/10.7910/DVN/YGLYDY>

[2] Berke, A., Calacci, D., Mahari, R., Yabe, T., Larson, K., & Pentland, S. Open e-commerce 1.0, five years of crowdsourced U.S. Amazon purchase histories with user demographics. *Scientific Data*, 11, Article 491, 2024.

<https://doi.org/10.1038/s41597-024-03329-6>

[3] Pandas Development Team. Pandas Documentation, 2026.

<https://pandas.pydata.org/docs/>

[4] Waskom, M. Seaborn Documentation, 2026.

<https://seaborn.pydata.org/>

[5] Scikit-learn Developers. Scikit-learn User Guide, 2026.

https://scikit-learn.org/stable/user_guide.html

[6] GDPR.eu. General Data Protection Regulation Overview, 2026.

<https://gdpr.eu/>

[7] State of California Department of Justice. California Consumer Privacy Act (CCPA), 2026.

<https://oag.ca.gov/privacy/ccpa>

[8] Association for Computing Machinery. ACM Code of Ethics and Professional Conduct.

<https://www.acm.org/code-of-ethics>